Room: IDOR on the Shelf

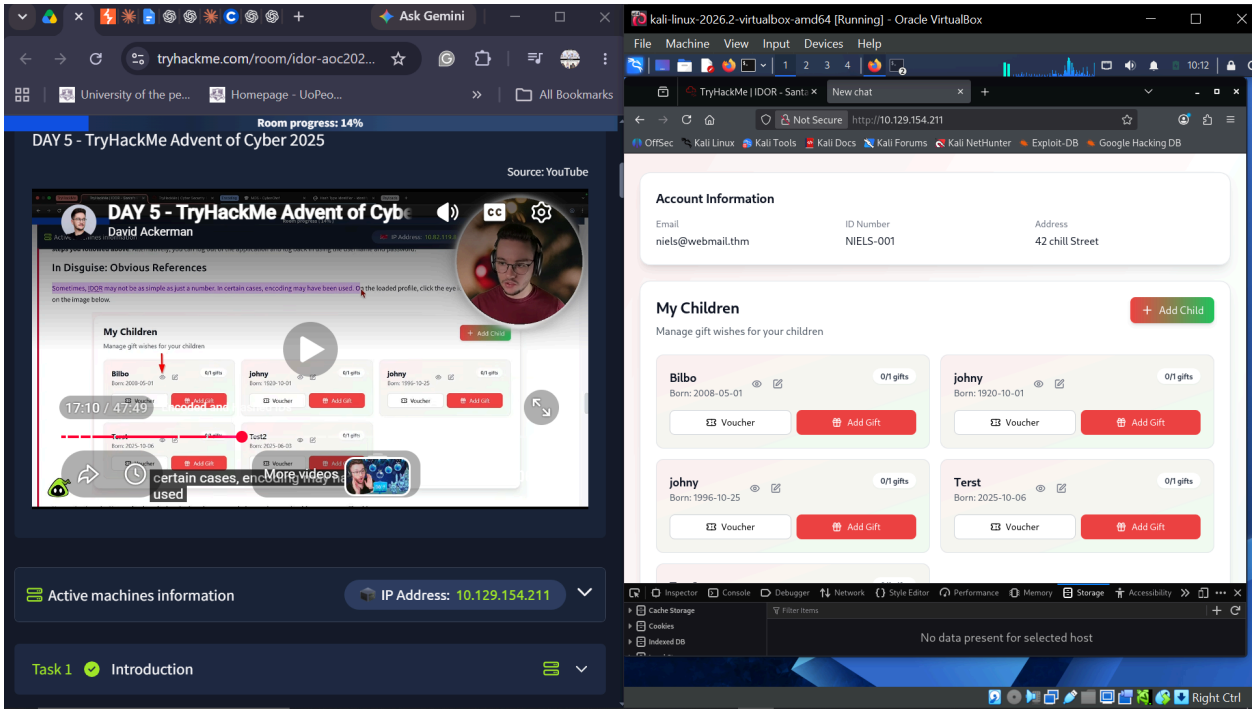
Platform: TryHackMe
Section: Task 4 - Common Web Vulnerabilities → **Authentication & Access Control**
Objective: Understand authentication vs. authorization; learn to identify and exploit Insecure Direct Object Reference (IDOR) vulnerabilities; and understand how to fix them (turning IDOR into "Secure Direct Object Reference"). **Reword the notes so they sound like you.**

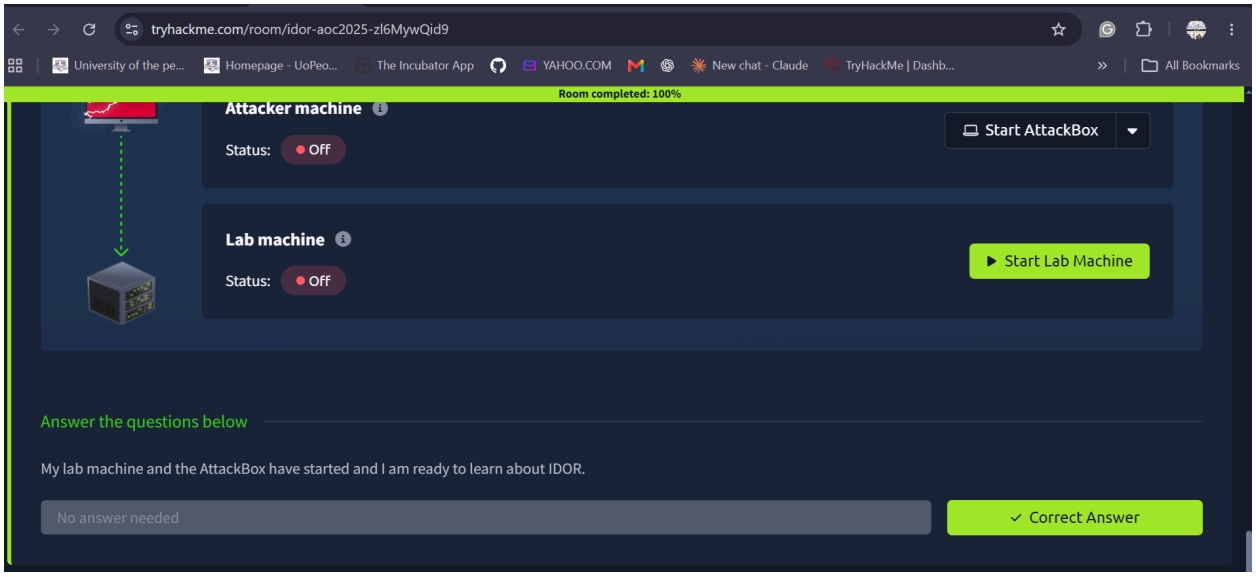
Task 1: Introduction

Notes:

> The scenario: parents can't activate vouchers on the TryPresentMe website, and targeted phishing emails are using non-public information, suggesting a data leak. Investigators found a suspicious account ("Sir Carrotbane") holding many vouchers. The goal is to find and fix the underlying vulnerability. Learning objectives: understand authentication/authorization, spot IDOR opportunities, exploit IDOR for horizontal privilege escalation, and learn to fix IDOR into a Secure Direct Object Reference (SDOR).

Screenshot:





Task 2: IDOR on the Shelf

Core concept

Notes:

> **IDOR (Insecure Direct Object Reference)** is an access control vulnerability. It happens when an application uses a direct reference (such as packageID=1001) to fetch data without verifying that the requester is actually authorised to view that specific record. Because these IDs are often sequential and guessable, an attacker can simply increment the number to access other users' data. The underlying query (SELECT ... WHERE packageID = value) has no concept of "does this belong to the current user"; it just returns whatever matches the ID.

Authentication vs Authorization

Notes:

- **Authentication:** verifying who you are (username/password). This isn't a one-time event; every request with your session cookie/token is re-authenticated.
- **Authorization:** verifying what you're allowed to do once authenticated. Authorization can only happen after authentication; if the app doesn't know who you are, it can't check your permissions.

Privilege escalation types

Notes:

- **Vertical privilege escalation:** gaining access to higher-level features (e.g. a normal user performing admin actions).
- **Horizontal privilege escalation:** staying at the same permission level but accessing another user's data (e.g. viewing someone else's account). **IDOR is usually horizontal**, since you're authorized to use the feature, just not for that specific record.

Practical exploitation: direct numeric IDOR

Notes:

I logged in as niels and used Developer Tools (Network tab) to inspect the view_accountinfo request, which used a user_id parameter (value 10, matching my own account). I then went to the **Storage/Application tab → Local Storage**, found the auth_user entry, and edited the user_id value directly in the browser (changing 10 to another number), then refreshed the page. This immediately loaded a completely different user's account data, with no further authentication required. This is the simplest form of IDOR: the server trusted a client-controlled value to decide whose data to return.

How I found the answer (user_id with 10 children):

I iterated through different user_id values in Local Storage, refreshing and checking how many children were listed for each account, until I found the account with exactly 10 children. The answer was **user_id = 15**.

Disguised IDOR: Base64 encoding

Notes:

Clicking the "view child" icon sent a request using a Base64-encoded value (e.g. Mg==) instead of a plain number. Decoding it (echo "Mg==" | base64 -d) revealed it was simply the number 2. Encoding doesn't prevent IDOR, it just adds a trivial extra step, since anyone can decode Base64 instantly. I could still perform the same IDOR attack by encoding whichever ID I wanted to test.

Disguised IDOR: hashed values

Notes:

The "edit child" request used a hash-like string (MD5/SHA1 style) instead of a plain ID. A hash looks random, but if the hashing algorithm and input format can be identified (using a hash identifier tool), an attacker can replicate the hash for any

target ID and still perform IDOR. Hashing an ID is not the same as properly authorising access to it.

Deepest case: UUID-based IDOR

Notes:

The voucher codes looked fully random but were actually **UUID version 1**, which encodes the exact timestamp of generation. If an attacker knows roughly when a voucher was created (e.g. “generated between 20:00-21:00”), they can generate every possible UUID for that time window (3600 combinations for one hour) and brute-force a valid voucher. This shows that even seemingly random identifiers can be predictable if the generation algorithm is flawed.

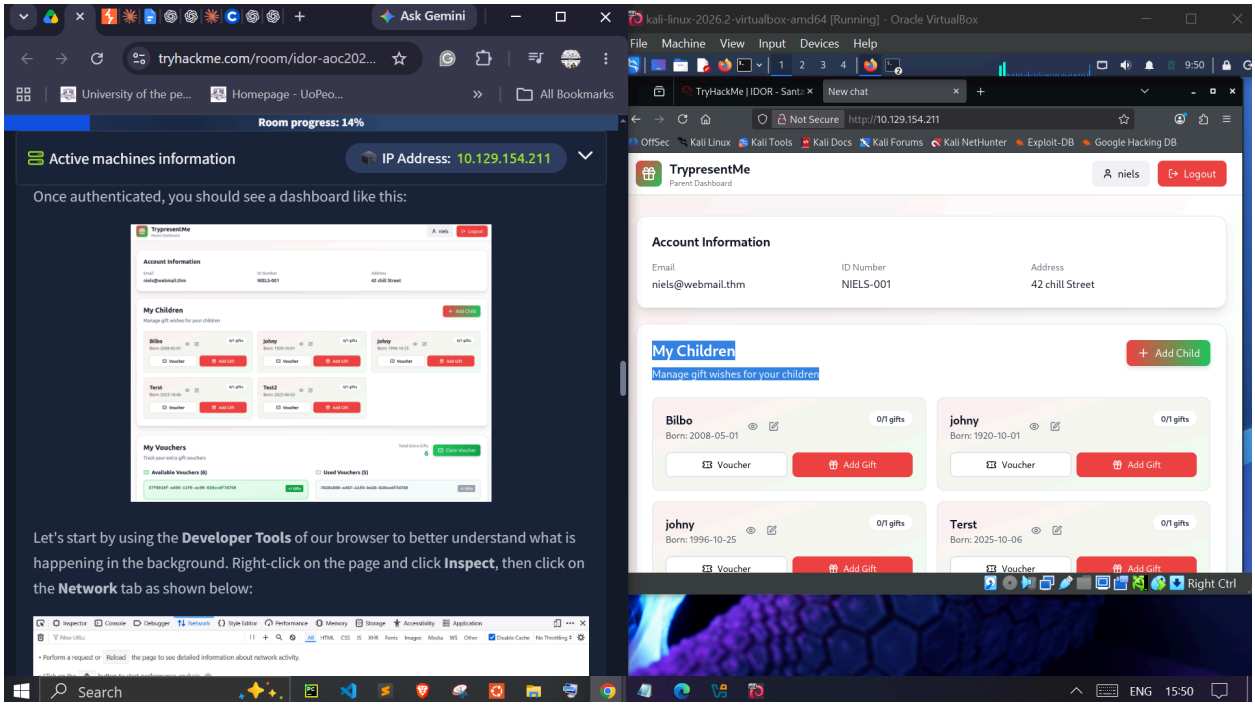
Fixing IDOR

Notes:

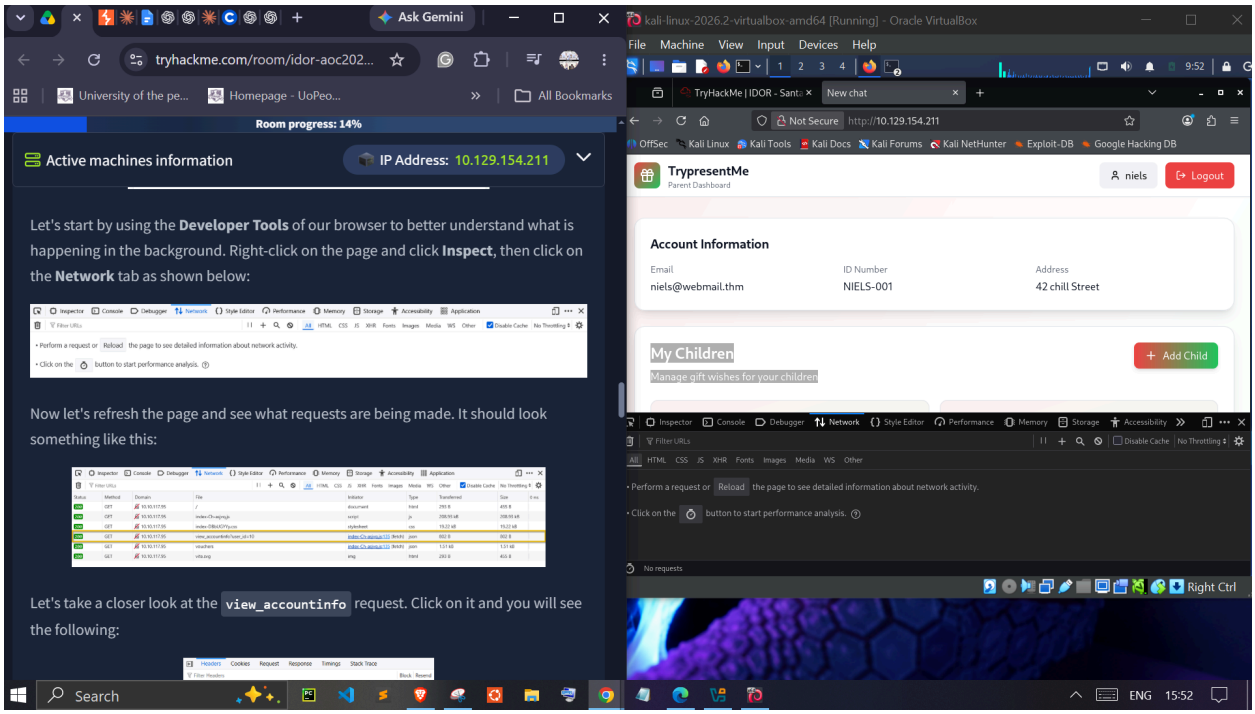
The real fix is **server-side authorization checks on every request**: “does this logged-in user own or have permission to access this specific item?” Obscuring IDs with Base64 or hashing is not a substitute for this check, since both can be reversed or replicated. Using random, hard-to-guess IDs helps reduce casual guessing but is not sufficient alone. Applications should also log and monitor failed access attempts, since repeated failures can indicate an active IDOR exploitation attempt.

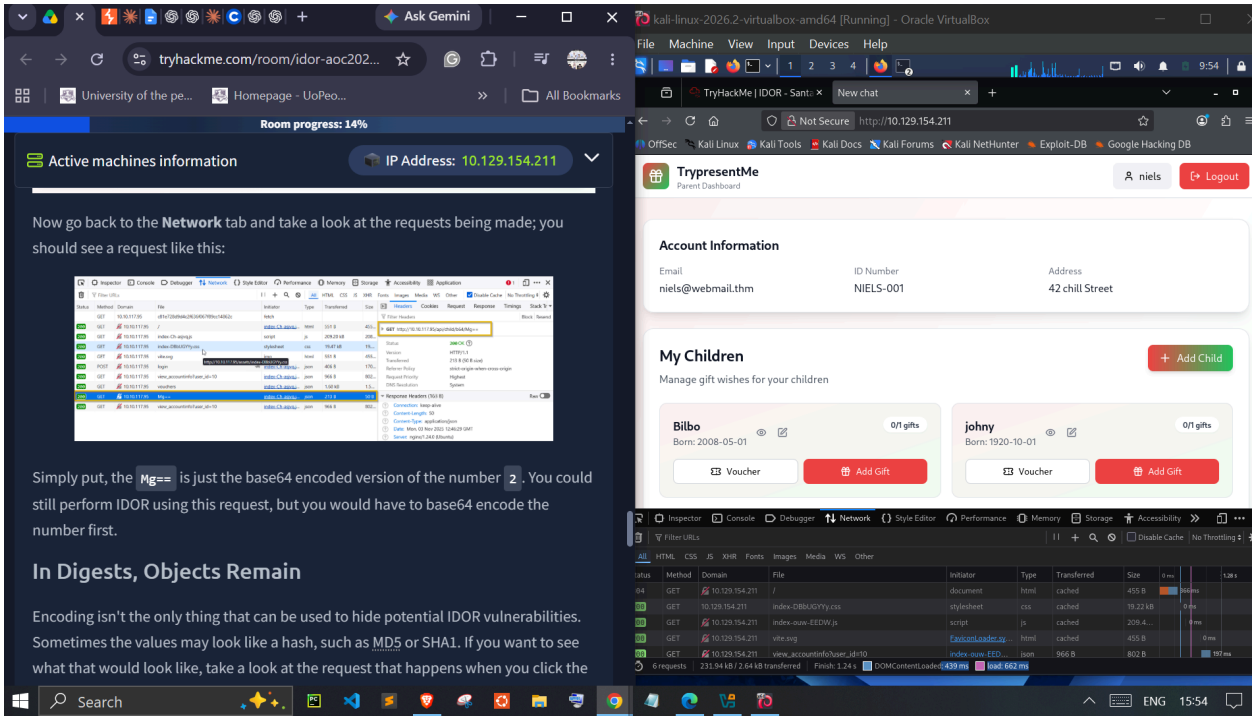
Evidence to save

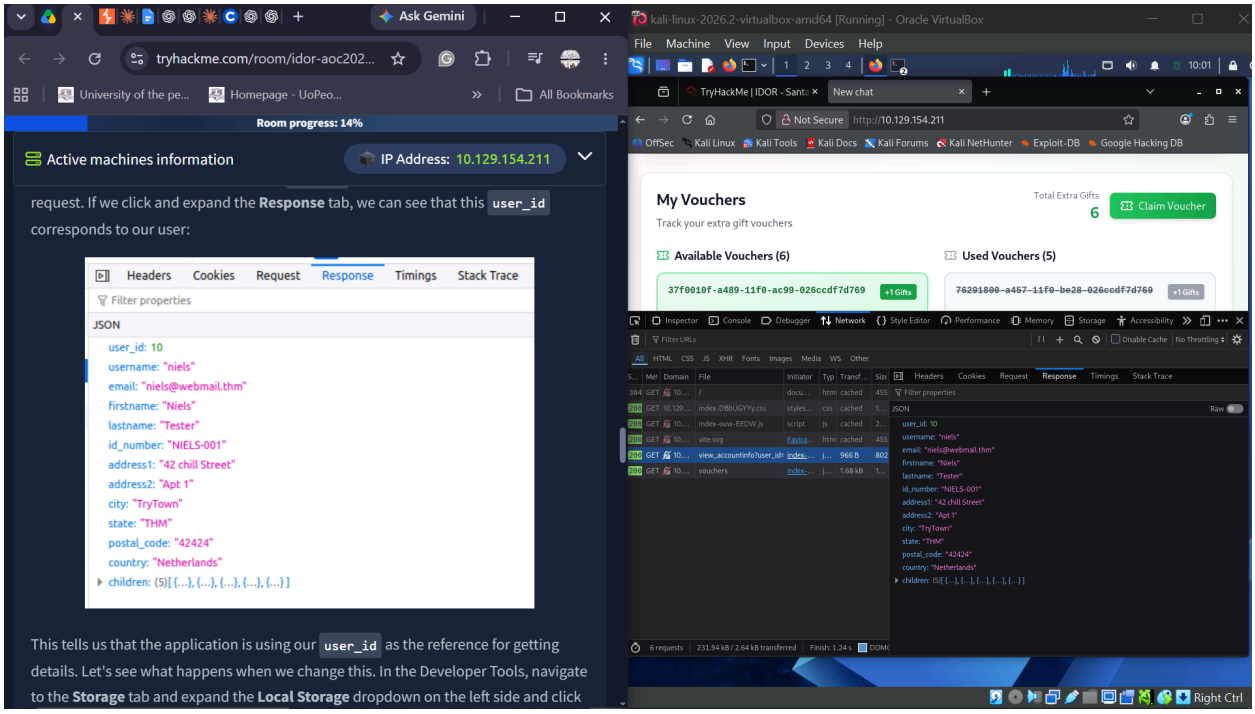
- Screenshot of login and dashboard



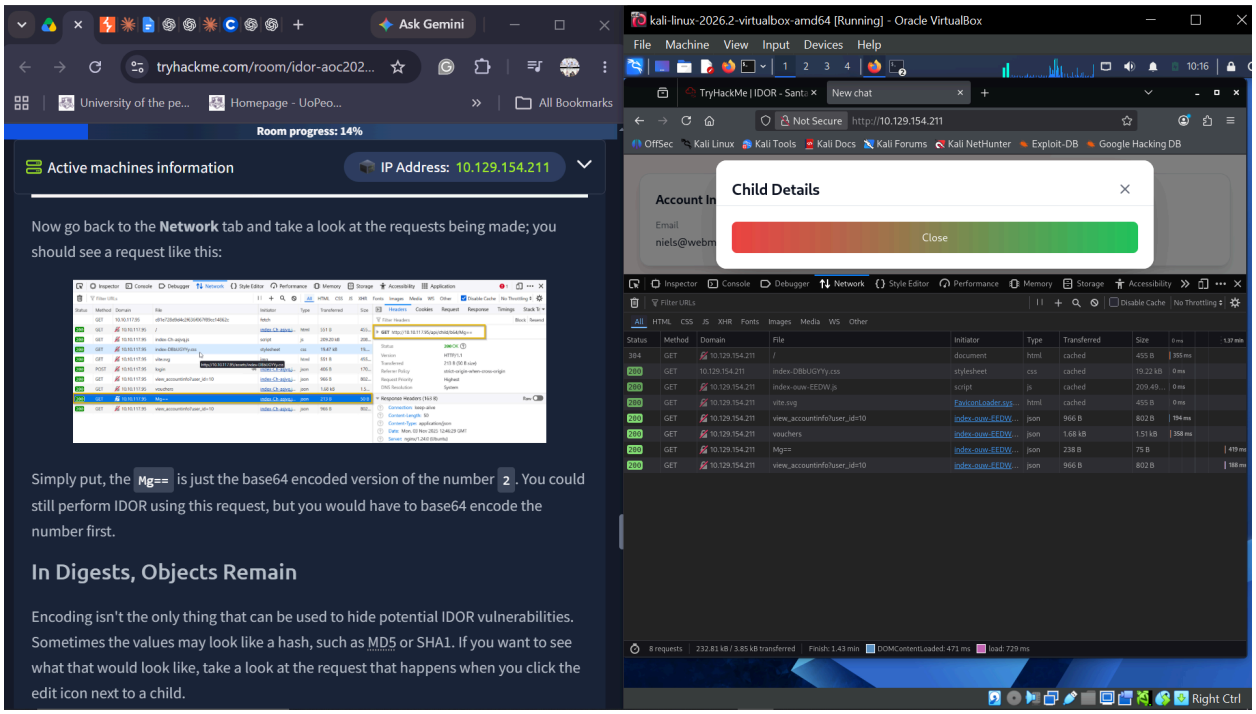
- Screenshot of Developer Tools showing view_accountinfo request with user_id: 10

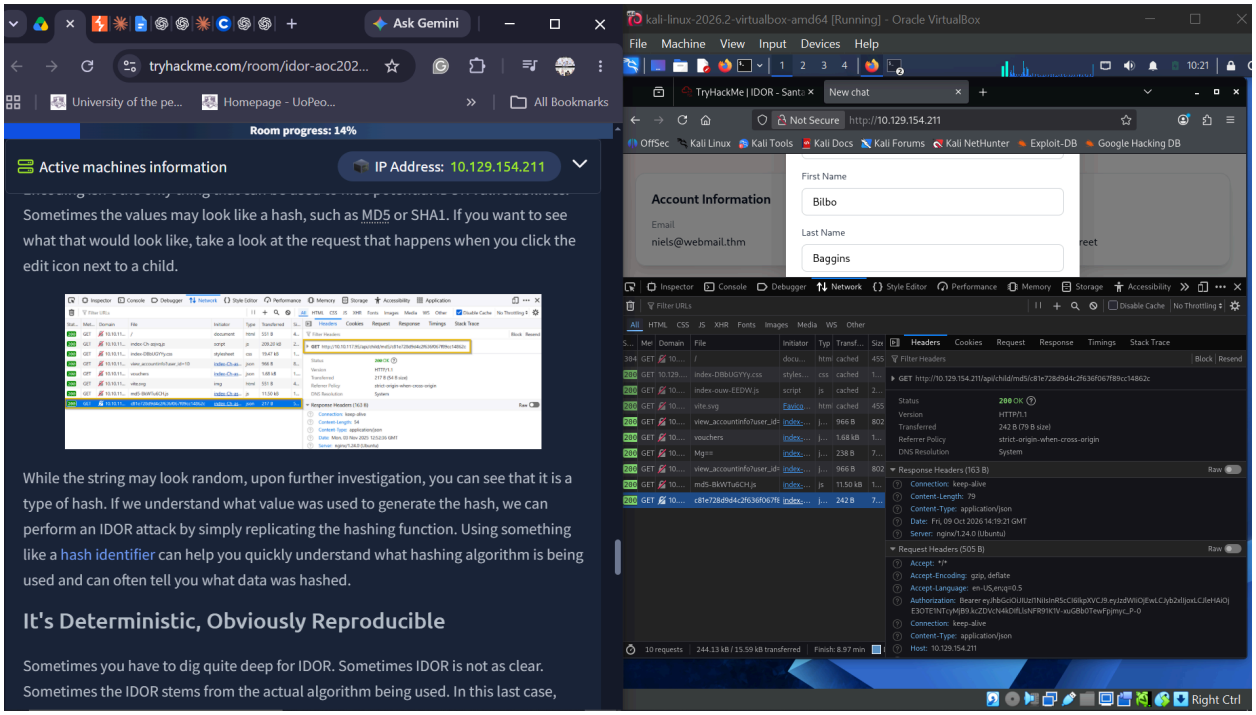




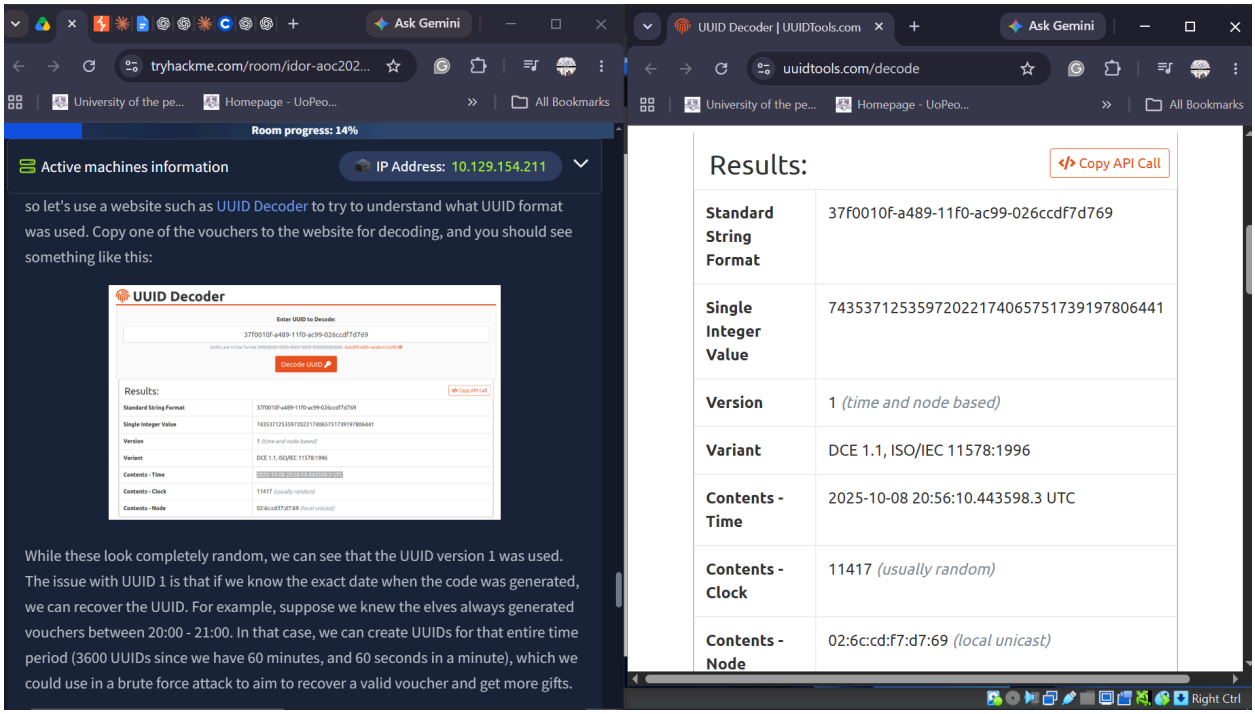


- Screenshots of the Base64 and MD5 child endpoint requests

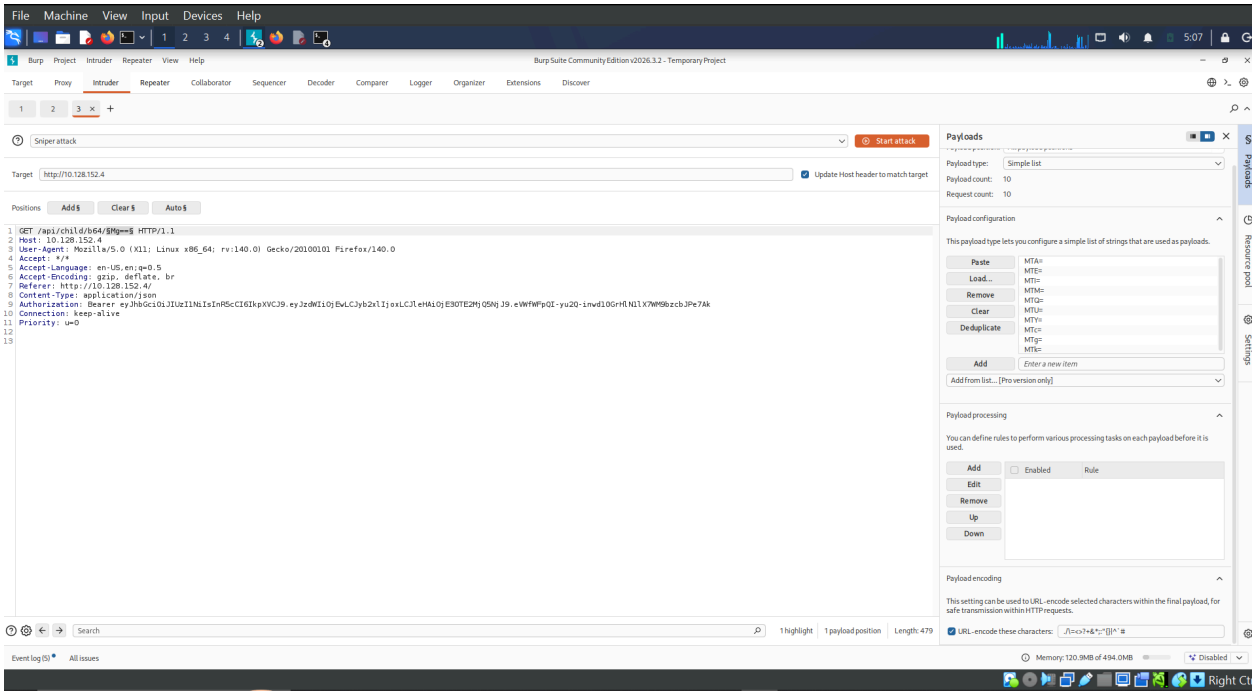


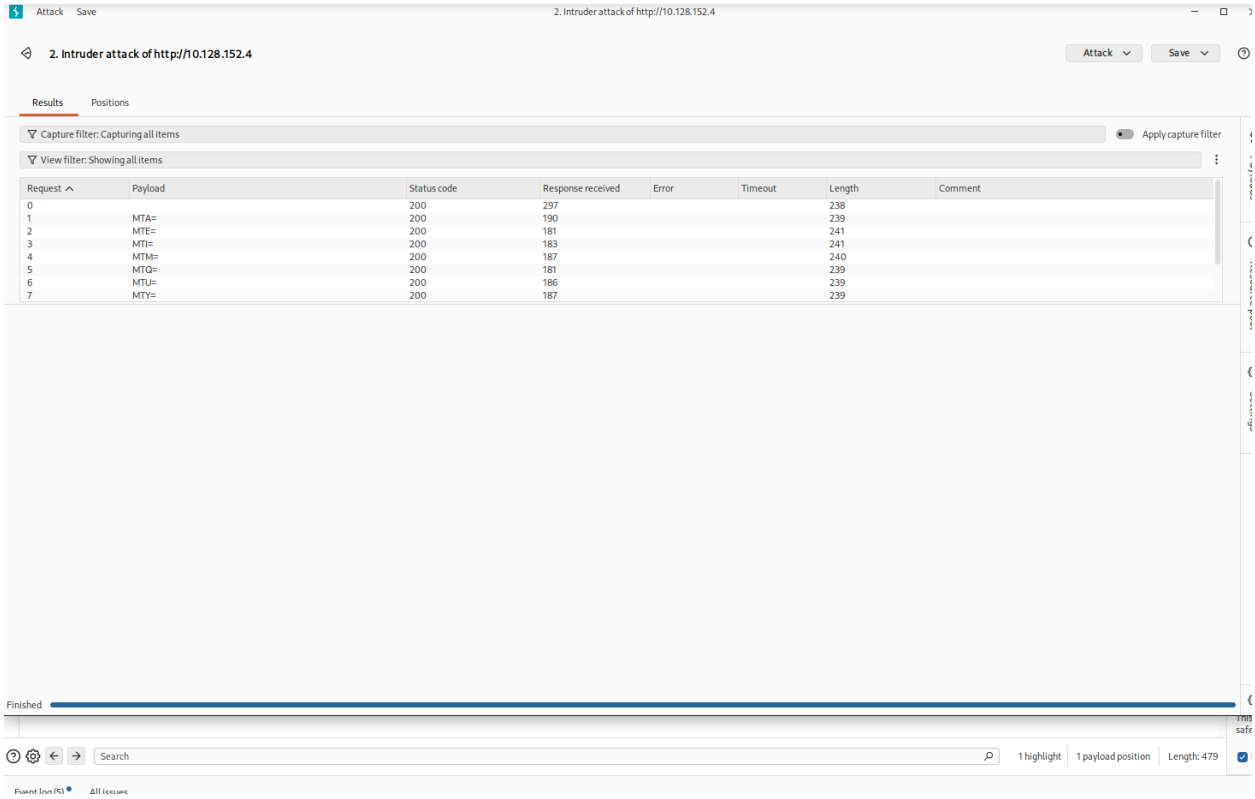


- Screenshot of the UUID decoder showing UUID v1

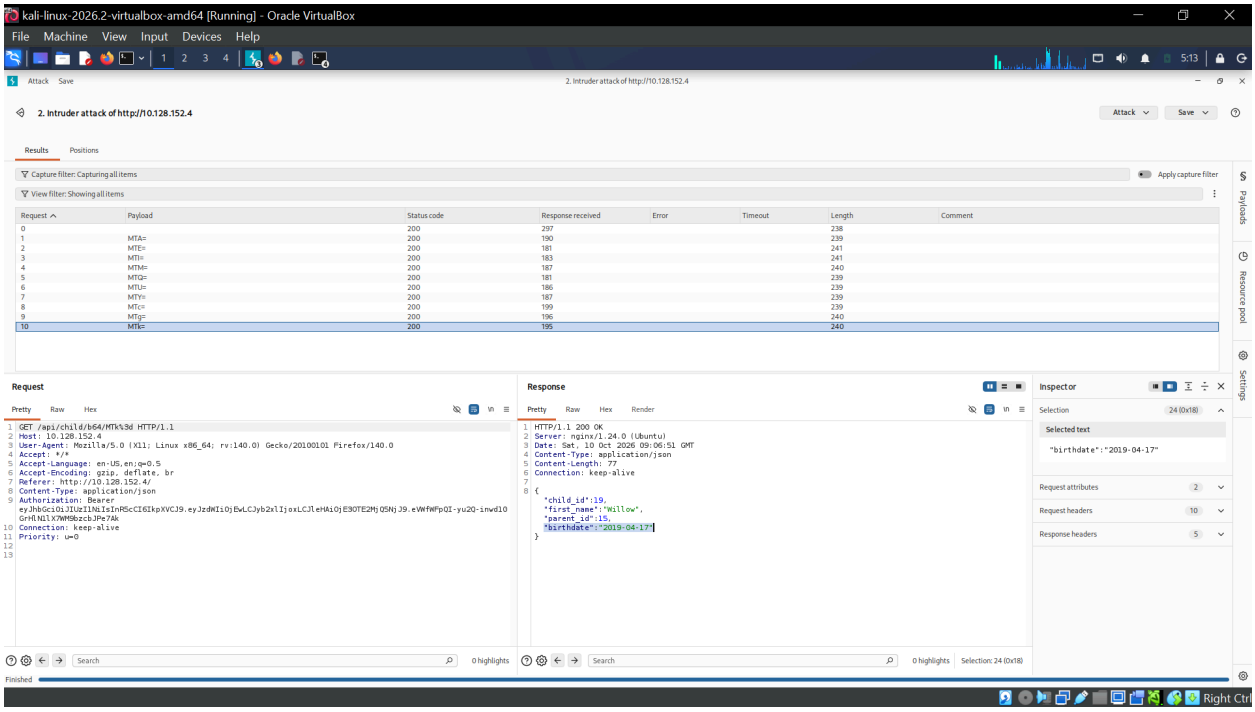


- Both core answers and the user_id answer ticked green

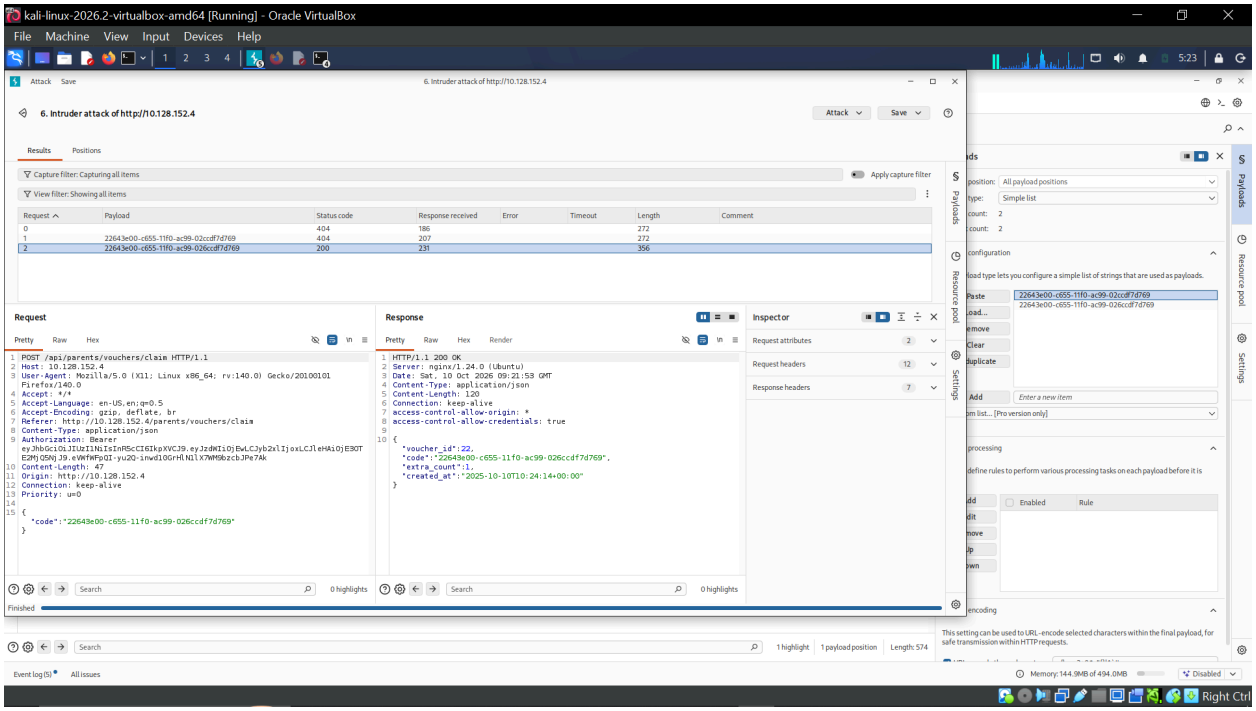


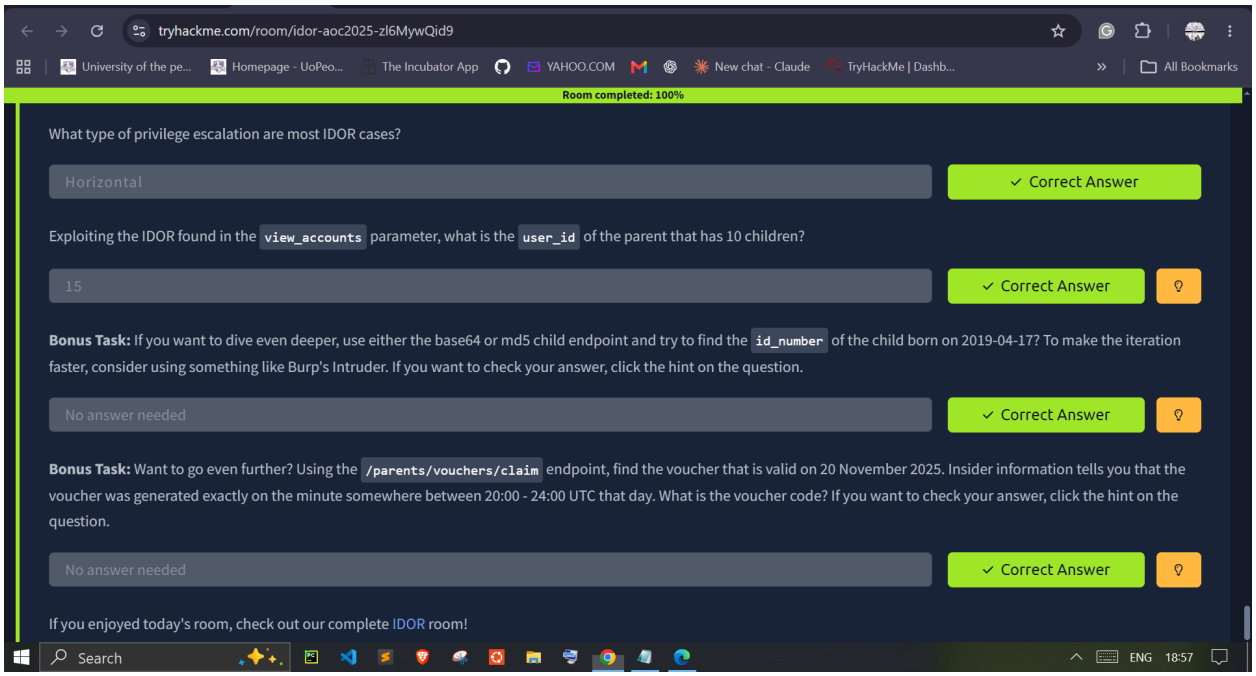


- Finding the child born on 2019-04-17



- Finding the voucher valid on 20 November 2025


kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
6. Intruder attack of http://10.128.152.4
Request
Response
Inspector
Right Ctrl


tryhackme.com/room/idor-aoc2025-zl6MywQid9
Room completed: 100%
What type of privilege escalation are most IDOR cases?
Horizontal
✓ Correct Answer
Exploiting the IDOR found in the view_accounts parameter, what is the user_id of the parent that has 10 children?
15
✓ Correct Answer
Bonus Task: If you want to dive even deeper, use either the base64 or md5 child endpoint and try to find the id_number of the child born on 2019-04-17? To make the iteration faster, consider using something like Burp's Intruder. If you want to check your answer, click the hint on the question.
No answer needed
✓ Correct Answer
Bonus Task: Want to go even further? Using the /parents/vouchers/claim endpoint, find the voucher that is valid on 20 November 2025. Insider information tells you that the voucher was generated exactly on the minute somewhere between 20:00 - 24:00 UTC that day. What is the voucher code? If you want to check your answer, click the hint on the question.
No answer needed
✓ Correct Answer
If you enjoyed today's room, check out our complete IDOR room!

Tools used

Browser Developer Tools (Network tab, Storage/Local Storage), Burp Suite Intruder (for the bonus Base64/MD5 brute-force), base64 -d, a UUID decoder website, Kali Linux / AttackBox

Security observations to include

- **Client-side storage (Local Storage, cookies) must never be trusted for authorization.** The application used a client-editable user_id value as its sole access control mechanism, which is a critical flaw.
- **Obfuscation is not security.** Base64 encoding and even hashing IDs only slow down an attacker slightly; neither replaces a real server-side permission check.
- **Predictable identifier generation is dangerous.** UUID version 1 embeds a timestamp, making it brute-forceable within a known time window, a case where the "randomness" of an ID was an illusion.
- **Real-world impact:** in the room's scenario, this class of vulnerability directly enabled sensitive data disclosure (parents' personal information) and voucher fraud (the "Sir Carrotbane" account).
- **Remediation:** enforce ownership/permission checks server-side on every single request, regardless of how the ID is formatted or hidden; use monitoring/alerting on failed or suspicious access attempts; prefer random, non-sequential IDs as a secondary hardening measure (not a replacement for authorization checks).

Final "What I learned" paragraph

> This room gave me a thorough, practical understanding of IDOR as an authorization failure rather than just "changing a number in a URL." I learned that IDOR can be disguised through encoding or hashing, and that even algorithmically-generated identifiers like UUIDs can be predictable if the generation method is flawed, as shown by brute-forcing a UUID version 1 voucher using a known time window. Working through the direct numeric IDOR, Base64, MD5, and UUID cases, each progressively more hidden, helped me understand that the common root cause is always the same: the server trusts a client-supplied reference instead of verifying the requester's actual permission to that specific record. The bonus tasks using Burp Intruder also gave me practical experience automating iteration through many possible ID values rather than testing them one at a time manually.

Completion Link:

https://tryhackme.com/room/idor-aoc2025-zl6MywQid9?utm_campaign=social_share&utm_medium=social&utm_content=share-completed-room&utm_source=copy&sharerId=6a298268d8b6b164348fab72

PortSwigger Lab: Username Enumeration via Different Responses

Platform: PortSwigger Web Security Academy
Section: Task 4 - Common Web Vulnerabilities → **Authentication & Access Control**
Level: Apprentice
Objective: Enumerate a valid username by comparing subtle differences in application responses, then brute-force that user's password, and use both to gain unauthorized access to their account. **Reword the notes so they sound like you.**

Background / Concept

Notes:

This lab demonstrates two chained authentication vulnerabilities:

1. **Username enumeration:** the application returns a different response (length and message) depending on whether a submitted username exists, even though the generic error message is supposed to look the same for both invalid usernames and valid usernames with wrong passwords. This "Invalid username" vs "Incorrect password" difference leaks which usernames are real.
2. **No brute-force protection:** once a valid username is known, the application allows unlimited password attempts with no account lockout or rate limiting, making automated password brute-forcing possible.

Combining both means an attacker can first confirm a real username exists, then systematically try every password in a wordlist against it, eventually gaining full account access.

Setup

Notes:

I accessed the lab through PortSwigger's own cloud infrastructure (no TryHackMe VPN needed, since this runs on the public internet). I used **Burp Suite Community Edition** inside my Kali VM, with Firefox's proxy configured to route traffic through

Burp (127.0.0.1:8080), and imported Burp's CA certificate into Firefox to allow HTTPS interception.

Screenshot:

Part 1: Enumerating the valid username

Steps performed:

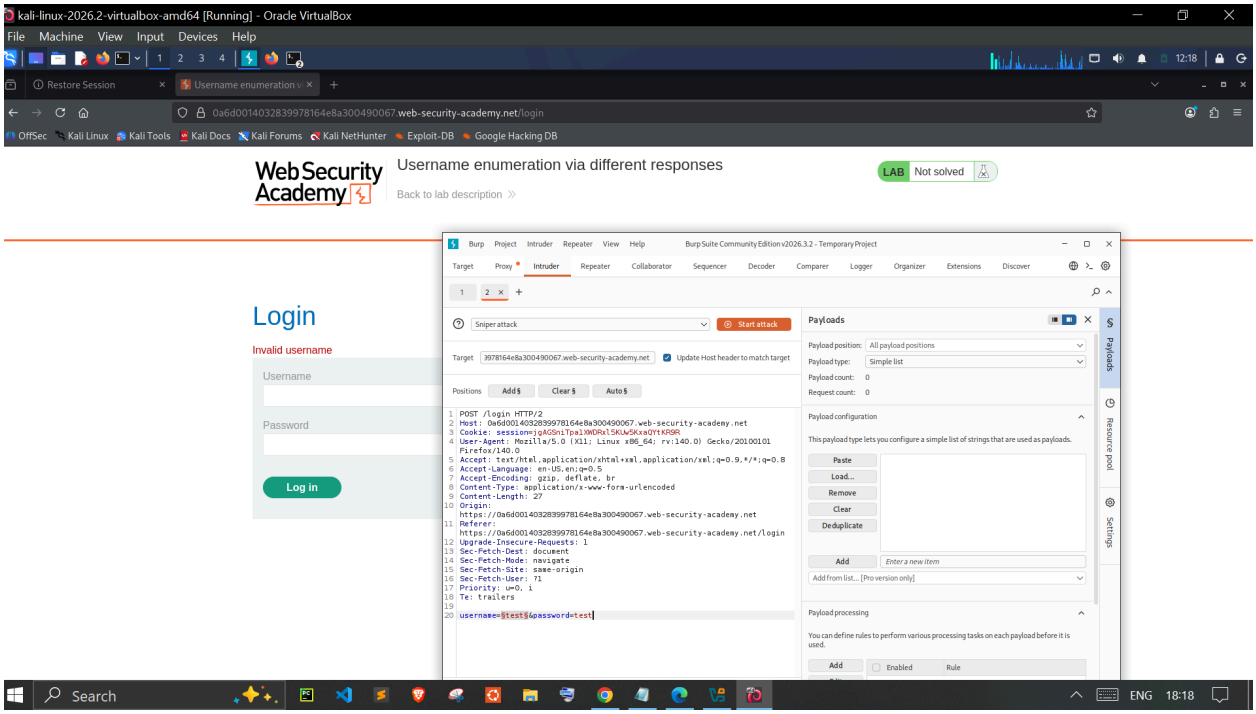
1. Captured a login request (POST /login) in Burp's Proxy with Intercept on, using placeholder credentials (test/test).
2. Sent the captured request to **Intruder**.
3. In the **Positions** tab, marked only the username parameter value as the payload position: username=§test§&password=test
4. Set the attack type to **Sniper**.
5. In **Payloads**, loaded the candidate usernames wordlist (from PortSwigger's own usernames list, cleaned and loaded via file to avoid blank-line issues from copy-paste).
6. Ran the attack.
7. Sorted the results table by the **Length** column. Nearly all responses shared the same length and displayed **"Invalid username"**. One response had a different length and displayed **"Incorrect password"** instead, revealing a valid username.

Notes:

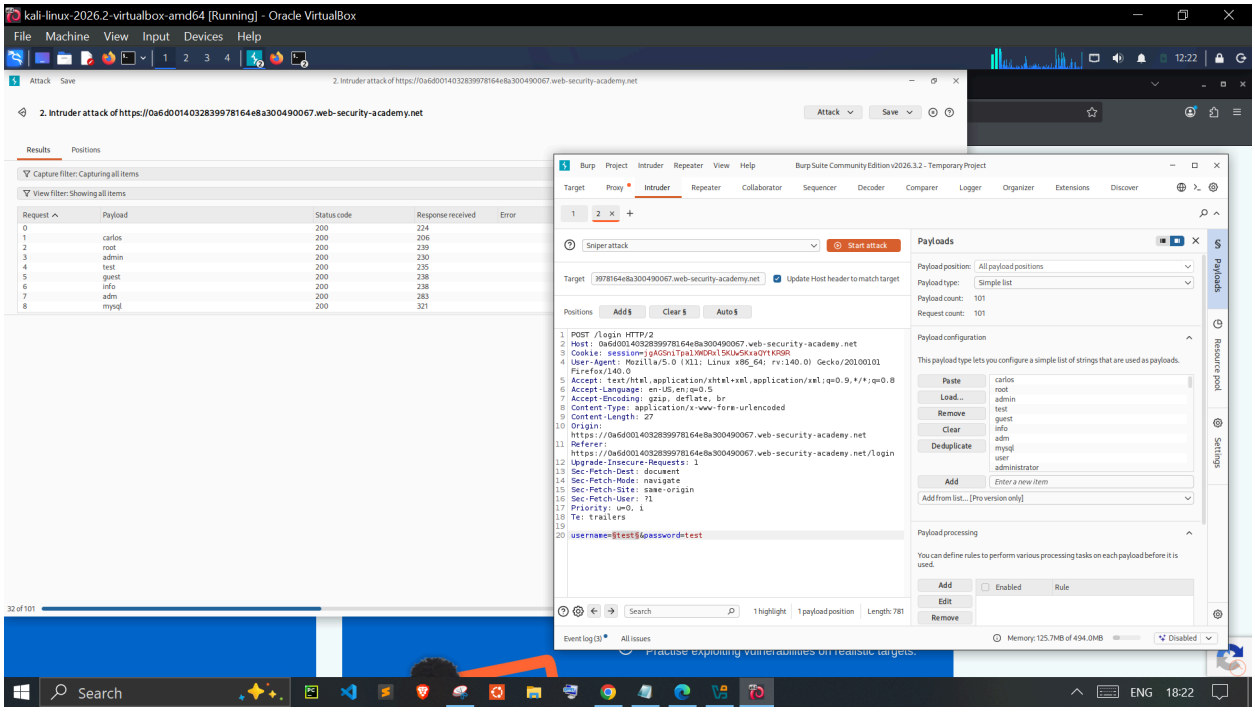
This is a classic username enumeration technique. Even though both error cases are meant to look identical to a user, the underlying code paths produced slightly different response content, and therefore different byte lengths, which Burp Intruder made trivial to spot at scale by comparing hundreds of responses automatically.

Screenshots:

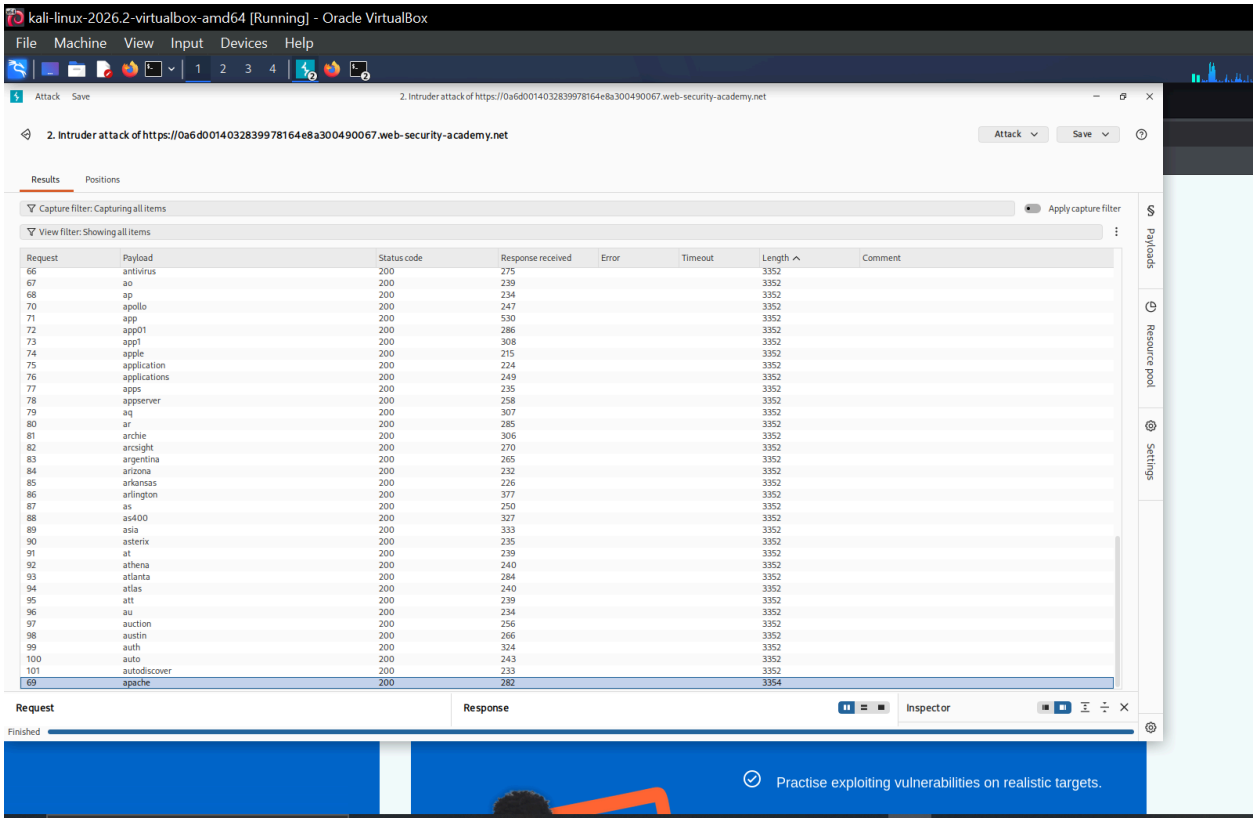
- Positions tab showing username=§test§&password=test



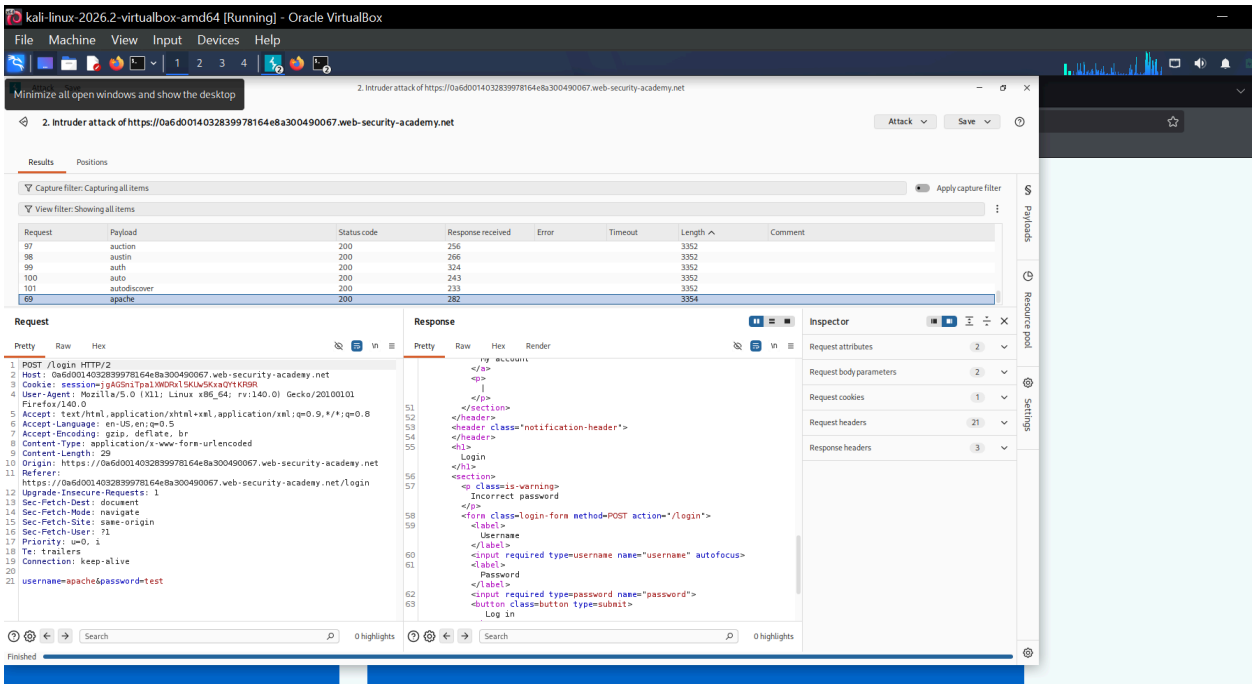
- Payloads tab showing the loaded usernames wordlist



- Results table sorted by Length, with the standout row highlighted



- The standout row's response showing "Incorrect password"



Part 2: Brute-forcing the password

Steps performed:

1. Captured a **fresh** login request (to ensure a valid, non-expired session), this time submitting the real username (azureuser) with a placeholder password.
2. Sent to Intruder, and in **Positions**, fixed the username as a static value and marked only the password parameter: username=azureuser&password=§test§
3. In **Payloads**, loaded the candidate passwords wordlist (same cleaning process as the usernames list).
4. Ran the attack.
5. Checked the **Status** column. Nearly all requests returned 200 (failed login, page reloads). **One request returned 302 Found**, a redirect indicating a successful login, with the response showing:

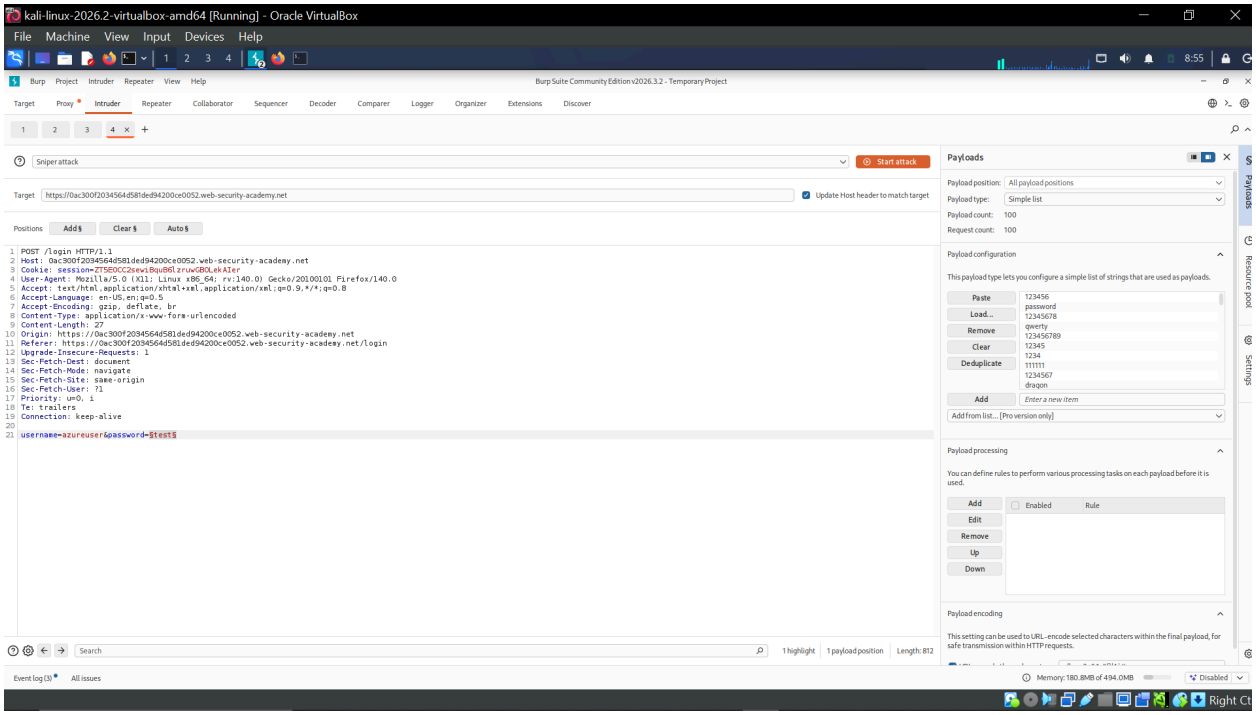
HTTP/2 302 Found

Location: /my-account?id=azureuser

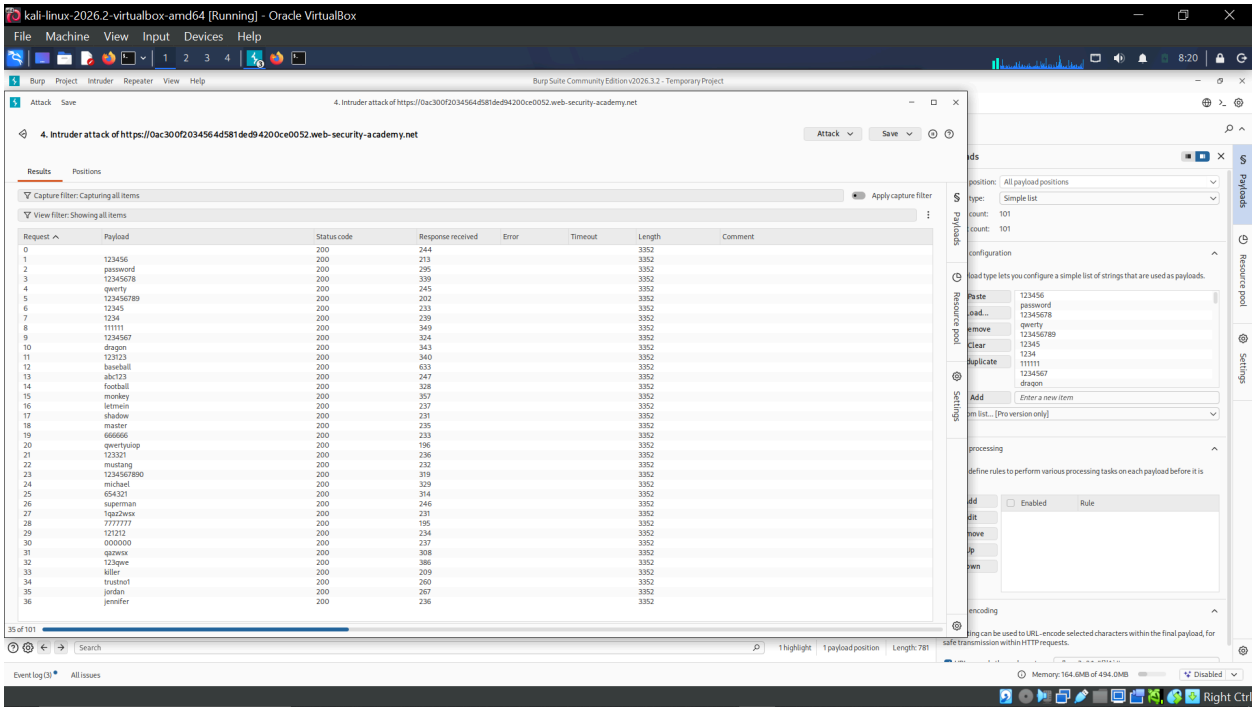
Set-Cookie: session=qbqZt7PTBa53zhWJnhXX8yf82Vm2FzZb; Secure; HttpOnly; SameSite=None

Screenshots:

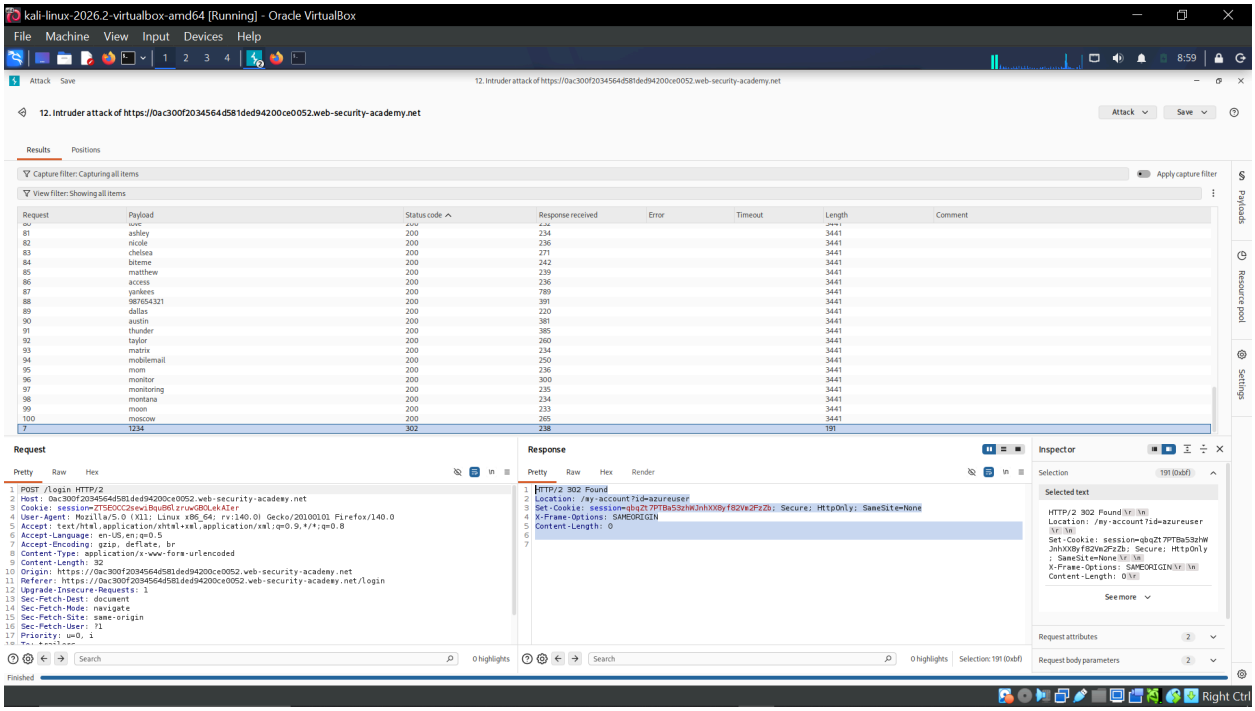
- Positions tab showing username=azureuser&password=§test§



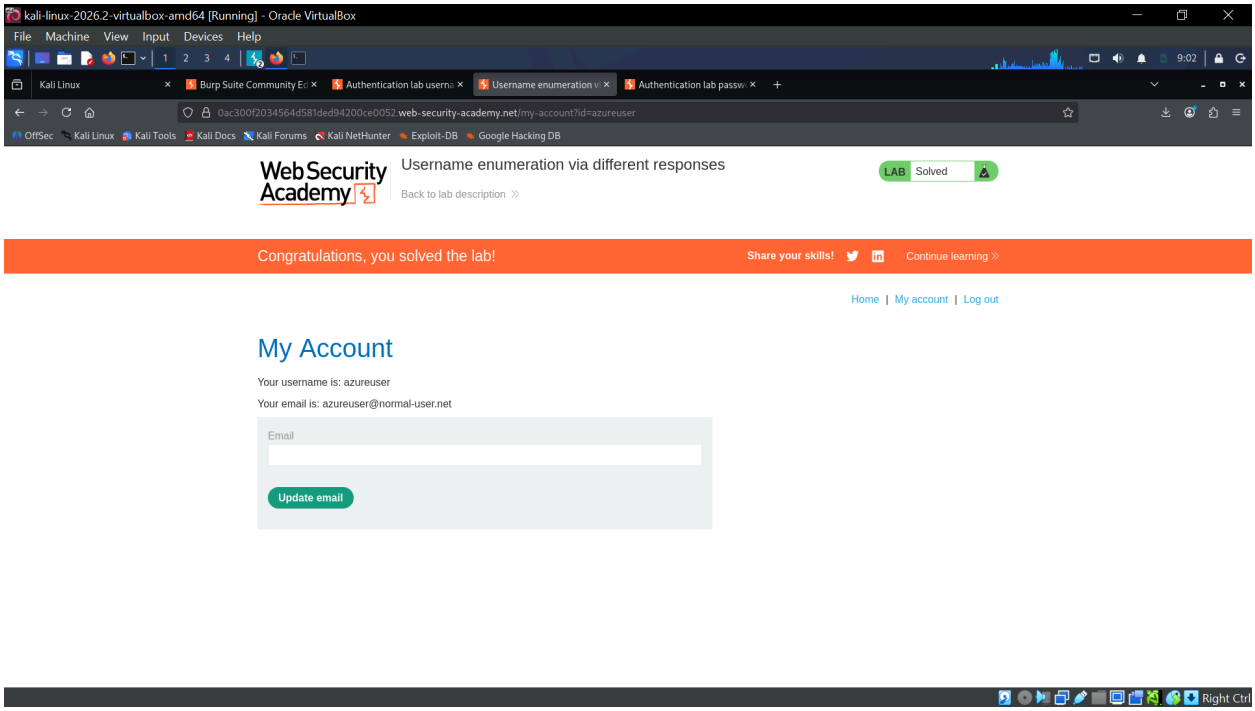
- Payloads tab showing the loaded passwords wordlist



- Results table showing the single 302 row among many 200s, and The 302 response itself (Location header and Set-Cookie)



- **The password**

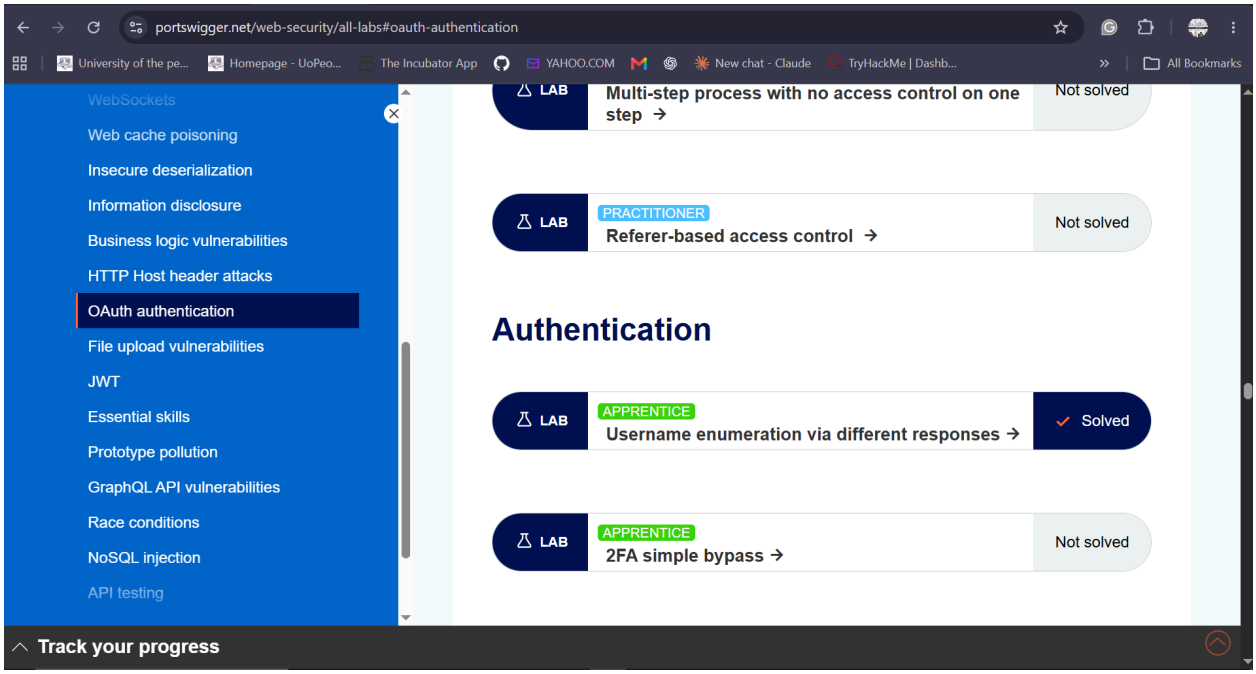


Part 3: Logging in and solving the lab

Steps performed:

1. Navigated to the lab's login page in the browser.
2. Entered username azureuser and the password identified from the password attack.
3. Clicked **Log in**.
4. Successfully landed on /my-account?id=azureuser.
5. The lab banner changed from "Not solved" to **"Solved"**.

Screenshots:



Problem-solving log (strong evidence for your report)

Notes:

This lab involved significant real-world troubleshooting beyond the core technique:

- **Payload position errors:** initially marked both username and password as payload positions simultaneously, causing malformed requests and 400 Bad Request responses. Fixed by using Clear § and re-marking only one field at a time.
- **Wordlist formatting issues:** copy-pasting the candidate wordlists directly from the browser introduced blank lines, causing 400 "missing parameter" errors for those entries. Resolved by pasting into a plain text file (nano) first to strip formatting, then loading the file into Burp via **Load...** instead of pasting directly.
- **Certificate trust error:** Firefox initially blocked the connection with an HTTPS warning ("Software is Preventing Firefox From Safely Connecting"), because Burp intercepts and re-signs HTTPS traffic with its own certificate. Resolved by importing Burp's CA certificate into Firefox's trusted authorities via http://burp.
- **Stale session errors:** after an extended troubleshooting session, requests began failing uniformly with connection errors ("Stream failed to close correctly") and identical responses across every payload. Diagnosed as an

expired session cookie and resolved by capturing a completely fresh request with a new session.
- **Lab instance change:** at one point the lab's domain itself changed (a new subdomain and session), meaning a previously captured request became entirely invalid. Resolved by confirming the current active URL and re-capturing against it.

Each issue was diagnosed by isolating one variable at a time (checking the request structure, then the wordlist, then the session, then the target itself) rather than changing multiple things at once, which made it possible to identify the actual root cause each time.

Tools used

Burp Suite Community Edition (Proxy, Intruder, Inspector), Firefox with Burp's CA certificate imported, Kali Linux, PortSwigger candidate username/password wordlists, nano

Security observations to include

- **Inconsistent error messages leak information.** Even generic-looking error pages can differ subtly enough (length, wording) to confirm whether a username exists.
- **No rate limiting or account lockout** allowed unlimited password attempts against a known-valid username, enabling full automated brute-forcing.
- **Remediation:** return identical, generic error messages (same length, same wording, same response time) regardless of whether the username or password was wrong; implement account lockout, CAPTCHA, or rate limiting after a small number of failed attempts; consider multi-factor authentication as an additional layer of protection.

Final "What I learned" paragraph

This lab gave me hands-on experience with a complete, realistic attack chain: using Burp Intruder to enumerate a valid username through subtle response differences, then brute-forcing that user's password by watching for a differing status code. Beyond the core technique, I gained significant practical experience in systematic troubleshooting, diagnosing malformed requests, cleaning wordlists properly, recognising and resolving HTTPS interception certificate issues, and identifying session expiry as the cause of uniform failures. This reinforced that real penetration testing involves as much careful debugging and environment troubleshooting as it does applying the core vulnerability technique itself.

Completion Link:

https://0ac300f2034564d581ded94200ce0052.web-security-academy.net/my-account?id=azureuser